

HTTP 协议 · 通俗讲解

浏览器和服务器的怎么"聊天"的 —— 用大白话把 HTTP 讲明白

JavaWeb 第四章 · HTTP 部分

目录

- | | |
|--------------------|-------------------|
| 1. HTTP 到底是什么 | 5. 报文长什么样 (重点) |
| 2. 发展历程 (扫一眼就行) | 6. 请求报文 (重点) |
| 3. 一次请求发生了什么 | 7. 响应报文 (重点) |
| 4. HTTP 1.0 vs 1.1 | 8. 状态码 (重点) |
| | 9. GET vs POST 总结 |

一、HTTP 到底是什么

HTTP 全称 **Hyper Text Transfer Protocol** (超文本传输协议)。名字很唬人，拆开就三件事：

- **超文本**：不只是纯文字，还能传网页、图片、视频、JSON..... 各种内容。
- **传输**：负责把数据从 A 点送到 B 点。
- **协议**：双方约定好的"规矩/格式"。

 **一句话理解**：HTTP 就是浏览器和服务器之间通信的"快递单格式"。你寄快递得按格式填单子（寄件人、收件人、地址），对方才能看懂；浏览器和服务器传数据也一样，得按统一格式写"报文"，对方才看得懂。

通俗讲：**你发请求 (request)**，**服务器回响应 (response)**，一来一回都遵循 HTTP 这个"格式规范"。它规定了：

- 浏览器发给服务器的东西，长什么样 → **请求报文**

- 服务器回给浏览器的东西，长什么样 → **响应报文**



HTTP 就是浏览器和服务器之间“对话”的规矩



一次请求、一次响应，有来有往

二、发展历程（扫一眼就行）

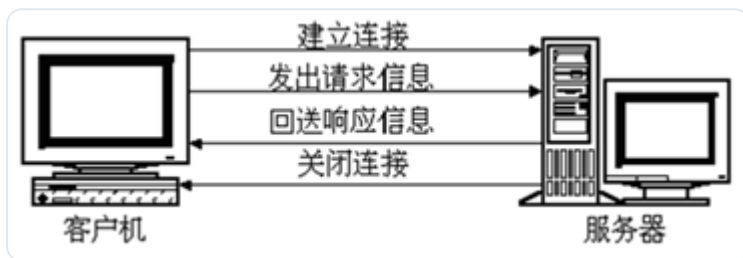
这部分不用背，知道“版本在升级、越来越快”就够了。真正用到的还是 1.1 和 2。

版本	时间	一句话特点
HTTP/0.9	1991	最早版本，只会返回一个网页，简陋到只有一行。
HTTP/1.0	1996	支持 GET/POST，有了报文头、状态码，但 不是官方标准 。
HTTP/1.1	1997	第一个标准化版本 ，支持长连接，用了 15 年，现在仍很主流。
HTTP/2	2015	多路复用 + 二进制 + 头部压缩，速度更快。
HTTP/3	2021	改用 QUIC（基于 UDP），更快更稳。

三、一次请求，到底发生了什么

浏览器访问一个页面，和服务器之间要经历**四个步骤**：

1. **建立连接**：浏览器先和服务器"握手"，搭起一条通道。
2. **发送请求**：浏览器把请求报文发给服务器。
3. **返回响应**：服务器处理完，把响应报文回给浏览器。
4. **断开连接**：事情办完，连接关闭。



浏览器与服务器通信的四个步骤

⚠ **关键理解**：这个过程是**短暂的、独立的**。每访问一个页面都要建立一次连接，浏览器和服务器之间的每一次请求响应都是**完全独立的**——服务器不会"记住"你上一个请求是谁、干了啥。

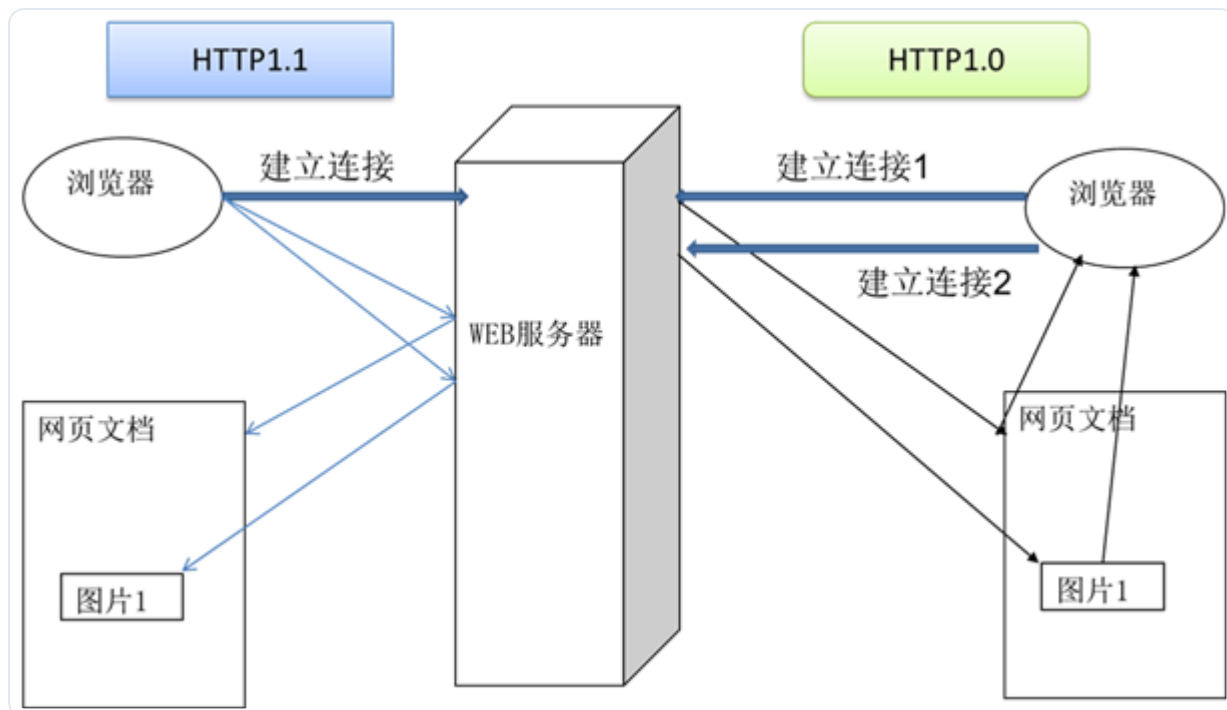
📦 **案例①**：你打开淘宝首页 —— 浏览器先和淘宝服务器握手建立连接，然后发"我要首页"的请求，服务器把首页的 HTML 回给你，连接断开。整个过程一次搞定。

📦 **案例②**：你再点进某个商品详情页 —— 这又是一个**全新独立**的连接。服务器并不知道"刚才你看了首页"，它只针对"我要这个商品"这个请求重新响应一次。这就是为什么需要 Cookie/Session 来"记住"你是谁（后面会学）。

四、HTTP 1.0 和 1.1 的区别

核心区别就一个字：**连接**。

- **HTTP 1.0 (短连接)**：请求一个带图片的网页，图片得**重新开一条新连接**去下载。
- **HTTP 1.1 (长连接)**：**一条连接**把当前请求的所有资源（文字、图片、样式）全传完，再断开。



1.0 每张图都要重新连一次，1.1 一条连接传到底

📦 **案例①：**HTTP 1.0 就像你网购 5 件商品，卖家分 5 次、每次单独发一个快递；1.1 就是卖家把你 5 件商品**打包成一个快递**一次发过来，明显更省事、更快。

📦 **案例②：**打开一个含 20 张图片的新闻网页：1.0 要建立 21 次连接（1 次文字 + 20 次图片），1.1 只需 1 次连接就把 21 样东西全拉回来 —— 效率高了一大截。

五、报文到底长什么样（重点）

浏览器和服务端之间传的数据，统称**报文 (Message)**。它的结构是固定的：

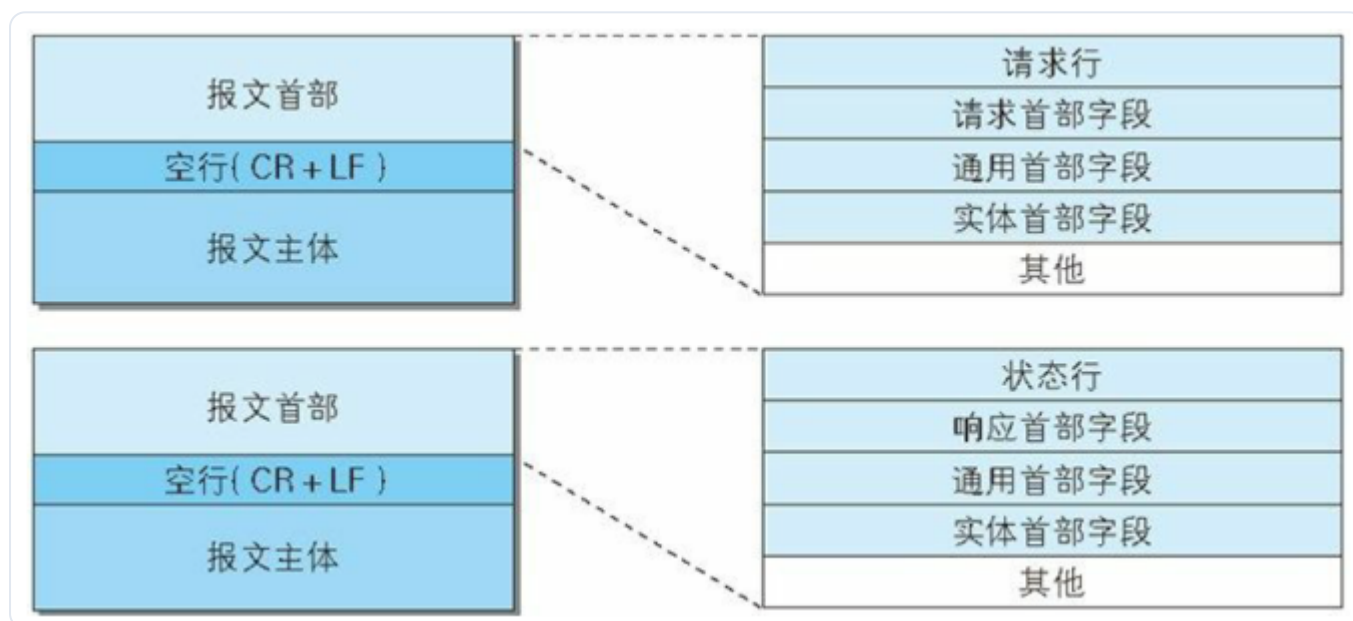
- **报文首部：**一些“说明信息”（这是啥、多长、什么类型……）
- **空行：**隔开首部和主体，是个“分界线”
- **报文主体：**真正要传的“内容”



报文 = 首部 + 空行 + 主体

其中，**报文首部**又能细分成两行：

- **行**：第一行，最重要（请求行 / 响应行）
- **头**：后面的若干行，一堆 key: value 的说明



报文首部 = 行 + 头

💡 **类比记忆**：报文就像一封信 —— **首部**是"信封"（写了收件人、寄件人、说明），**主体**是"信纸"（真正的内容），中间那行空行就是"信封和信纸的分界"。

🔍 **怎么看真实报文？** 浏览器按 **F12** 打开开发者工具，切到 **Network (网络)** 标签，刷新页面，点任意一条请求，就能看到它完整的请求头和响应头。这是你学后面 Servlet/SpringMVC 时天天要用的技能。

浏览器窗口显示地址为 `localhost:8080/web01_war_exploded/`，页面内容为 `hello web01`。

下方展示了 Chrome DevTools 的网络（Network）面板，显示了请求 `web01_war_exploded/` 的详细信息。请求方法为 `GET`，状态码为 `304`（Not Modified）。请求头（Request Headers）包括 `Accept`、`Accept-Encoding`、`Accept-Language` 和 `Cache-Control`。响应头（Response Headers）包括 `Connection`、`Date`、`ETag` 和 `Keep-Alive`。

名称	标头	预览	响应	发起程序	计时
web01_war_exploded/	请求 URL: <code>http://localhost:8080/web01_war_exploded/</code> 请求方法: <code>GET</code> 状态代码: <code>304</code> 远程地址: <code>[::1]:8080</code> 引用者策略: <code>strict-origin-when-cross-origin</code>				
favicon.ico					

2 次请求 已传输 131 B 21.8 kB

用 F12 → Network 抓取真实的请求响应报文

案例①：把报文想象成寄信 —— 首部是信封（收件人地址、快递方式、说明），主体是信纸（你要说的真正内容）。看一封信先看信封，看一个报文也先看首部。

 **案例②：**想知道网页为什么打不开？打开 F12 的 Network，点那个红色（失败）的请求，看看返回的**状态码**是 404 还是 500，就能立刻判断"是网址错了"还是"服务器自己挂了"。这比瞎猜强一百倍。

六、请求报文（重点）

请求报文 = 浏览器发给服务器的数据。格式固定为：

```
请求行（第一行）
请求头（若干行 key: value）
<空行>
请求体（POST 才有）
```

6.1 请求行

第一行，由三部分组成：**请求方式** + **资源路径** + **协议版本**。

```
GET /05_web_tomcat/login_success.html?username=admin&password=123213 HTTP/1.1
```

- GET —— 请求方式
- /05_web_tomcat/login_success.html —— 要访问的资源路径
- ?username=admin&password=123213 —— 跟在网址后面的参数
- HTTP/1.1 —— 协议及版本

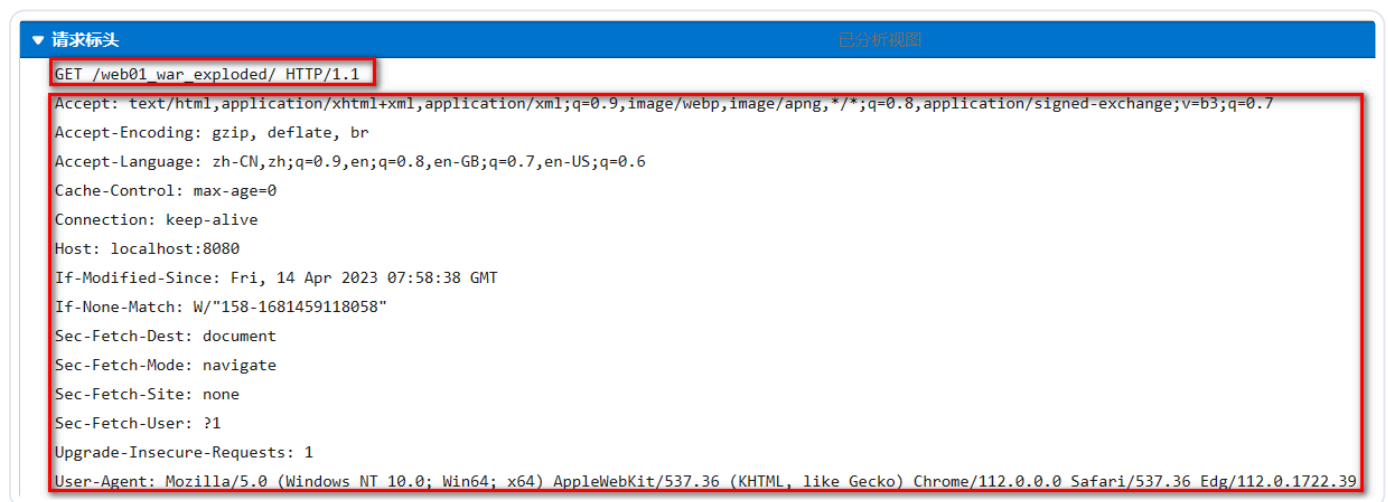
6.2 请求头

一串 **名字：值** 的说明，告诉服务器一些额外信息：

```
// 主机虚拟地址（访问的是哪个服务器）
Host: localhost:8080
// 长连接
Connection: keep-alive
// 浏览器（客户端）的系统信息
User-Agent: Mozilla/5.0 (Windows NT 6.1; ...) Chrome/68.0 ...
// 浏览器支持的文件类型
Accept: text/html,application/xml;q=0.9,image/webp,*/*;q=0.8
// 从哪个页面跳转过来的（用于防盗链、广告计费）
Referer: http://localhost:8080/05_web_tomcat/login.html
// 浏览器支持的压缩格式
Accept-Encoding: gzip, deflate, br
// 浏览器支持的语言
Accept-Language: zh-CN,zh;q=0.9,en-US;q=0.8,en;q=0.7
```

6.3 请求体

只有 POST 请求才有请求体，里面放的是真正提交的数据。GET 的数据直接跟在网址后面，没有请求体。



F12 里看到的请求报文（请求行 + 请求头 + 请求体）

6.4 GET 请求的特点

form 表单默认提交方式就是 GET。它就像一张明信片：

- 参数拼在网址后面，地址栏可见，不安全
- 地址栏大小有限（一般 4KB），只能带少量文本
- 没有请求体，所以封装和解析都快、效率高

- 地址栏输入回车、超链接、表单默认提交，都是 GET

6.5 POST 请求的特点

POST 就像把东西塞进一个**包裹箱**：

- 数据放在**请求体**里，地址栏看不到，**相对安全**
- 请求体大小**没有限制**，能传大文件、图片
- **上传文件只能用 POST**
- 多了一些和请求体相关的请求头（如 `Content-Type`、`Content-Length`）
- 比 GET **效率略低**

✦ **POST 怎么设置**：把 form 标签的 `method` 属性改成 `post` 即可（默认是 `get`）。

```
<form action="target.html" method="post">
  用户名: <input name="username" type="text" />
  <input type="submit" value="提交" />
</form>
```

form 表单把 `method` 设为 `post`

📦 **案例① (GET)**：你在百度搜索"Java"—— 关键词就明文出现在网址里 `...?wd=Java`。好处是快、还能收藏这个网址；坏处是你搜了什么，旁边的人看地址栏就知道。

📦 **案例② (POST)**：你在网页登录、输密码点登录 —— 密码是放在**请求体**里发给服务器的，地址栏干干净净看不到。这就是为什么登录、注册、传文件这类"敏感/大"的数据，一律用 POST。

七、响应报文（重点）

响应报文 = 服务器回给浏览器的数据。格式和请求报文对称：

响应行（第一行）
响应头（若干行 `key: value`）
<空行>
响应体（真正的内容，比如 HTML）

7.1 响应行

由三部分组成：**协议版本** + **状态码** + **状态描述**。

```
HTTP/1.1 200 OK
```

- `HTTP/1.1` —— 协议版本
- `200` —— 状态码（请求成功）
- `OK` —— 状态描述

7.2 响应头

```
Server: Apache-Coyote/1.1           // 服务器版本信息
Content-Type: text/html              // 响应体数据类型（浏览器据此解析）
Content-Length: 157                 // 响应体字节数
Date: Mon, 13 Aug 2018 02:47:57 GMT  // 响应时间（可能有 8 小时时差）
ETag: W/"157-1534126125811"
Last-Modified: Mon, 13 Aug 2018 02:08:45 GMT
```

7.3 响应体

真正要给浏览器看的内容，比如一段 HTML 页面：

```
<!-- 响应体：浏览器拿到后会显示成页面 -->
<!DOCTYPE html>
<html>
  <body>
    恭喜你，登录成功了...
  </body>
</html>
```

▼ 响应头

```
HTTP/1.1 200
Accept-Ranges: bytes
ETag: W/"158-1681459118058"
Last-Modified: Fri, 14 Apr 2023 07:58:38 GMT
Content-Type: text/html
Content-Length: 158
Date: Sat, 15 Apr 2023 02:22:03 GMT
Keep-Alive: timeout=20
Connection: keep-alive
```

F12 里看到的响应报文（响应行 + 响应头）



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Title</title>
6 </head>
7 <body>
8   <h1>hello web01</h1>
9
10 </body>
11 </html>
```

响应体 —— 浏览器最终显示的页面内容

📦 **案例①：**你登录成功后，服务器回一个响应报文，响应体里装的是"恭喜登录成功"那段 HTML，浏览器把它渲染成你看到的页面。

📦 **案例②：**如果响应体是一个 `Content-Type: text/html` 的网页，浏览器会显示它；但如果响应的是一个 `.exe` 文件，浏览器不会去"显示"，而是弹出一个**下载框**。这就是响应头 `Content-Type` 的作用 —— 告诉浏览器"这是什么类型，该怎么处理"。

八、状态码（重点）

状态码就是服务器处理完请求后，告诉浏览器"结果如何"的一个数字。它非常重要，一看数字就能判断问题出在哪。

8.1 先记住这 6 个

状态码	含义	大白话
200	请求成功	一切正常，浏览器把内容显示出来。
302	重定向	服务器说"你换个地址再来"，并告诉你新地址（Location 头）。
304	使用了本地缓存	内容没变，浏览器直接用本地缓存，省流量。
404	资源没找到	你请求的网址不存在（客户端请求错了）。
405	请求方式不允许	比如该用 POST 你却用 GET。
500	服务器内部错误	资源找到了，但服务器代码自己出错了。

 **案例① (404)**：你手滑把网址打错一个字母，服务器找不到这个资源，回你 **404**，浏览器显示"页面不存在"。这是**客户端**的问题（你地址打错了）。

 **案例② (500)**：网址是对的，但服务器后台代码写错了（比如除零、空指针），服务器自己崩了，回你 **500**。这是**服务器**的问题（程序员写错代码）。记住：404 怪你，500 怪后端。

8.2 状态码分类

状态码第一位数字就代表它的"类别"：

开头	类别	含义
1xx	信息	服务器收到请求，需要继续。
2xx	成功	请求被成功处理（200 最常见）。
3xx	重定向	需要进一步操作才能完成（302、304）。
4xx	客户端错误	客户端请求有问题（404、405）。
5xx	服务器错误	服务器处理出错（500）。

 **案例①（302 重定向）**：你访问一个旧网址，服务器回 **302**，并带上 `Location: 新地址` 头，浏览器就自动跳转到新地址。就像公司搬家，门口贴了张“我们搬去 XX 号了”。

 **案例②（304 缓存）**：你第二次打开同一个网页，服务器发现内容没变，回 **304**，浏览器直接用上次缓存的页面，不用重新下载 —— 所以第二次打开明显更快。

8.3 完整状态码速查表

码	英文	中文含义
100	Continue	继续，客户端应继续请求。
101	Switching Protocols	切换协议。
200	OK	请求成功。
201	Created	成功并创建了新资源。
202	Accepted	已接受，但未处理完。
204	No Content	成功处理，但无内容返回。
206	Partial Content	成功处理了部分 GET 请求。
301	Moved Permanently	资源永久移动（永久重定向）。
302	Found	资源临时移动（临时重定向）。
303	See Other	查看其它地址。
304	Not Modified	资源未修改，用缓存。
307	Temporary Redirect	临时重定向。
400	Bad Request	客户端请求语法错误。
401	Unauthorized	需要身份认证。
403	Forbidden	服务器理解请求，但拒绝执行。
404	Not Found	找不到资源。
405	Method Not Allowed	请求方法被禁止。
408	Request Time-out	服务器等待超时。
409	Conflict	请求发生冲突。
410	Gone	资源已永久删除。
413	Request Entity Too Large	请求实体过大。
414	Request-URI Too Large	URI 过长。

415	Unsupported Media Type	不支持的媒体格式。
500	Internal Server Error	服务器内部错误。
501	Not Implemented	服务器不支持该功能。
502	Bad Gateway	网关/代理收到无效响应。
503	Service Unavailable	服务器超载或维护，暂时不可用。
504	Gateway Time-out	网关超时。
505	HTTP Version not supported	不支持的 HTTP 版本。

九、GET vs POST 总结

对比项	GET	POST
数据位置	拼在网址后面	放在请求体里
安全性	不安全（地址栏可见）	相对安全
数据大小	受限（约 4KB）	无限制
请求体	没有	有
上传文件	不可以	可以（只能用 POST）
效率	高	略低
典型场景	搜索、超链接、地址栏输入	登录、注册、上传文件

 **本章小结：** HTTP 就是浏览器和服务器的对话“规矩”。你只需牢牢掌握三件事：① **报文格式（首部 + 空行 + 主体）** ② **请求报文和响应报文的结构** ③ **状态码（尤其 200/302/304/404/405/500）**。这三样是后面 Servlet、SpringMVC 的基石，天天都要打交道。